

Assignment 2

Writing Debugging and Testing Functions

Objective: Build a small console-based support-ticket analyzer. The application must use explicit success and failure results, lazy enumeration, pattern matching, a read-only data model, and interchangeable function strategies. Students must also debug two faulty functions, test the behavior, and run one summary operation in a separate thread.

1 Task

Implement a console application named FunctionalSupportDesk. Estimated workload: 90 to 120 minutes.

The program processes support tickets written in the following format:

```
id|priority|title|estimatedHours
```

Use the following input data. You may add more records for testing.

```
string[] rawTickets =  
{  
    "101|High|Cannot sign in|2",  
    "102|Low|Change profile photo|1",  
    "bad|Critical|Payment service unavailable|4",  
    "104|Unknown|Printer problem|2",  
    "105|Medium|3",  
    "106|Critical|Database unavailable|5"  
};
```

2 Required behavior

1. Parse every raw ticket without throwing an exception for expected invalid input.
2. Return both successful and failed parsing results as ordinary data.
3. Enumerate parsing results one at a time with `IEnumerable<T>` and yield return.
4. Use pattern matching to choose an action for every valid ticket.
5. Calculate a ticket score through a function passed as a strategy.
6. Correct the two faulty functions supplied in Section 4.
7. Run the final summary operation in one separate thread, then wait for it to finish.

3 Technical requirements

3.1 Read only model

- Create TicketPriority with Low, Medium, High, and Critical values.
- Represent each ticket with a readonly struct named SupportTicket.
- Provide read-only properties for Id, Priority, Title, and EstimatedHours.
- Do not use public mutable fields or setters.

3.2 Results oriented parsing

Create a pure parsing function with the following role. The exact result type may differ:

```
static (bool Success, SupportTicket? Value, string Error)
ParseTicket(string text)
```

- A ticket is valid only when the ID is positive, the priority exists, the title is not empty, and estimated hours are from 1 through 8.
- A failure must contain a clear message and must not use an exception for these expected validation problems.

3.3 Enumerable iterator

Create an iterator that processes the input sequence:

```
static IEnumerable<(bool Success, SupportTicket? Value, string Error)>
ParseTickets(IEnumerable<string> lines)
```

- Use yield return exactly once for every input line.
- Do not create a result List inside ParseTickets.
- Do not replace the required iterator with LINQ.

3.4 Pattern matching

Implement GetAction(SupportTicket ticket) with a switch expression or property patterns:

CRITICAL	Any	Escalate immediately
HIGH	4 or more	Assign senior specialist
HIGH	Less than 4	Prioritize
MEDIUM	Any	Standard queue
LOW	Any	Low priority queue

3.5 Strategy pattern

Pass scoring behavior into a calculation instead of placing every scoring rule inside that calculation:

```
static int CalculateScore(
    SupportTicket ticket,
    Func<SupportTicket, int> strategy)
```

- Create one priority-first strategy: Critical 100, High 60, Medium 30, Low 10.
- Create one quick-fix strategy that uses the priority-first score and adds 20 points when EstimatedHours equals 1.
- Demonstrate that CalculateScore works with either strategy without changing its code.

3.6 Functional separation

- Parsing, validation, action selection, scoring, and workload calculation must be deterministic functions.
- These functions must not read from or write to Console and must not modify input collections.
- Keep Console input and output in Main, PrintSummary, or another boundary function.
- Do not use mutable global state.

4 Debugging tasks

Copy the following functions into the project. Each contains one logical error. Correct both errors and write one short README sentence explaining each fix.

```

static bool IsUrgent(SupportTicket ticket)
{
    return ticket.Priority == TicketPriority.Critical
        && ticket.Priority == TicketPriority.High;
}

static int CalculateTotalHours(IEnumerable<SupportTicket> tickets)
{
    int total = 0;

    foreach (var ticket in tickets)
        total = ticket.EstimatedHours;

    return total;
}

```

5 Thread requirement

After processing the tickets, start one Thread that calls the final summary function. Call Start and Join. The summary thread may read the completed ticket collection, but it must not modify shared data. Multiple threads, locks, tasks, and synchronization primitives are not required.

```

var summaryThread = new Thread(() => PrintSummary(validTickets));
summaryThread.Start();
summaryThread.Join();

```

6 Expected summary

For the provided six input lines, the completed program must report:

VALID TICKETS	3
INVALID TICKETS	3
URGENT VALID TICKETS	2
TOTAL ESTIMATED HOURS	8

7 Required tests

Document at least eight tests in the README. The following six cases are mandatory; choose two additional boundary cases yourself:

101 HIGH CANNOT SIGN IN 2	Successful parsing
BAD CRITICAL SERVICE UNAVAILABLE 4	Invalid ID failure
104 UNKNOWN PRINTER PROBLEM 2	Invalid priority failure
105 MEDIUM 3	Empty title failure
HIGH TICKET	IsUrgent returns true
TICKETS WITH 2, 1, AND 5 HOURS	Total equals 8

8 README questions

1. Which functions are pure, where do side effects remain, and which C# features support the functional style?
2. Why does ParseTicket return a failure value instead of throwing for invalid input?
3. When does the code inside ParseTickets actually execute?
4. How does the Strategy pattern allow the scoring behavior to change?
5. What does readonly prevent in SupportTicket?
6. Why does Main call Join after starting the summary thread?

9 Deliverables

- A complete C# console project that compiles and runs.
- A README with run instructions, debugging explanations, answers to Section 8, and at least eight documented tests.
- One archive named IT-25XX_Surname_Name_Assignment2.zip uploaded through LMS by the announced deadline.
- Oral defense of the submitted code.

10 Assessment policy

- Maximum score: 15 points.

- The program must compile and run during the mandatory defense. A student must explain the submitted code and answer questions about any part of it.
- AI tools may be used only when explicitly permitted by the instructor and must not replace the student's understanding.
- Late submission conditions follow the official course syllabus and LMS announcement.
- Unexplained copied or generated code may receive zero according to the course academic-integrity policy.

CORRECTNESS AND RESULTS	Correct output and clear success or failure values	1
ENUMERABLE ITERATOR	Correct IEnumerable<T>, yield return, and lazy execution	2
MODEL AND MATCHING	Read-only model and correct matching actions	2
STRATEGY AND SEPARATION	Interchangeable strategy and separated side effects	2
DEBUGGING AND TESTS	Both defects explained and at least eight tests	2
THREAD	Summary thread starts and joins safely	1
ORAL DEFENSE	Student explains and predicts code behavior	5
TOTAL		15

Mandatory Grading Conditions

The following conditions override the criterion scores:

- No oral defense – 0/15. Submission of the project alone is not sufficient. Unless an alternative defense has been officially approved, a student who does not complete the oral defense receives zero for the assignment.
- The project does not compile – maximum 5/15.
- The project compiles but cannot correctly process the provided test data – maximum 7/15.
- If the student cannot explain a section of the submitted code, the corresponding assessment criterion receives 0 points.
- A required language feature that is only declared but does not meaningfully contribute to the program receives no credit.
- If yield return is explicitly required, replacing the iterator with a ready-made LINQ solution results in 0 points for the Enumerable Iterator criterion.
- Unexplained copied or AI-generated code results in 0/15 for the assignment.
- If several grading limitations apply, the lowest applicable maximum grade is used.